MLOPS / PEOPLE COUNTING

End-to-End MLOps: People Counting

A connected example with CO2, PIR motion and Python files

We predict the number of people in a meeting room using two inputs: CO2 concentration and PIR motion activity. We follow the same small model from data collection to deployment and monitoring.

FILES / INPUTS / OUTPUTS

Project: people_counting_mlops_simple/

WHY THIS STEP MATTERS

Synthetic data is for learning the workflow, not scientific validation.

CODE / TERMINAL COMMANDS

```
Input: co2_ppm, pir_motion
Target: people_count (0 to 8)
```

NEXT → See the complete journey first.

MLOPS / PEOPLE COUNTING

The whole journey

One project, many tools

First we collect and prepare data, then train and evaluate. Git and DVC preserve versions; MLflow records experiments. Airflow schedules runs; GitHub Actions tests and publishes. Flask and Docker serve the model.

FILES / INPUTS / OUTPUTS

Files: data/, src/, dvc.yaml, app.py, Dockerfile

WHY THIS STEP MATTERS

Each tool solves a specific problem in one lifecycle.

CODE / TERMINAL COMMANDS

```
DATA -> PREPARE -> TRAIN -> EVALUATE
-> TRACK -> AUTOMATE
-> SERVE -> DEPLOY -> MONITOR
```

NEXT → Set up the environment so scripts run consistently.

MLOPS / PEOPLE COUNTING

Project folder map

Know where every file belongs

Python scripts are in src/. The CSV lives in data/raw/. The trained model is created in models/. Configuration files in the project root connect the MLOps tools.

FILES / INPUTS / OUTPUTS

Root: people_counting_mlops_simple/

WHY THIS STEP MATTERS

Generated folders include data/processed/, models/, metrics/ and mlruns/.

CODE / TERMINAL COMMANDS

```
data/raw/people_count.csv
src/generate_data.py, prepare.py
src/train.py, evaluate.py, predict.py
dvc.yaml, params.yaml, app.py
Dockerfile, docker-compose.yml
.github/workflows/ci.yml, cd.yml
```

NEXT → Start by installing the libraries.

MLOPS / PEOPLE COUNTING

1. Environment setup

Conda + requirements.txt

Conda creates an isolated Python environment. requirements.txt lists the libraries the ML scripts need. Use VS Code to open and edit the project.

FILES / INPUTS / OUTPUTS

File: requirements.txt

WHY THIS STEP MATTERS

One environment prevents dependency conflicts.

CODE / TERMINAL COMMANDS

```
conda create -n people-mlops python=3.11 -y
conda activate people-mlops
pip install -r requirements.txt
code .
```

NEXT → Generate the first dataset.

MLOPS / PEOPLE COUNTING

2. Generate sensor data

Synthetic CSV with two features

The generator creates 240 example observations. Each row has CO2 ppm, PIR motion and a known people count. The label is needed to teach the model.

FILES / INPUTS / OUTPUTS

Python: src/generate_data.py | Output: data/raw/people_count.csv

WHY THIS STEP MATTERS

Real data would come from sensors plus independently collected ground truth.

CODE / TERMINAL COMMANDS

```
python -m src.generate_data
co2_ppm,pir_motion,people_count
420,0,0
510,2,1
780,8,5
```

NEXT → Inspect and prepare the CSV.

MLOPS / PEOPLE COUNTING

3. Read and inspect the CSV

Pandas reads the dataset

Read the CSV and check the feature names before training. Our scripts expect exactly the two input features and one label.

FILES / INPUTS / OUTPUTS

Input: data/raw/people_count.csv

WHY THIS STEP MATTERS

Good data quality matters more than a complex model.

CODE / TERMINAL COMMANDS

```
import pandas as pd
df = pd.read_csv("data/raw/people_count.csv")
print(df.head())
print(df.isna().sum())
```

NEXT → Split into training and testing sets.

4. Prepare train and test sets

Keep evaluation observations separate

The starter prepare.py reads the raw CSV and creates a 75% training split and 25% test split. The model must not train on the test rows.

FILES / INPUTS / OUTPUTS

Python: src/prepare.py | Outputs: data/processed/train.csv, test.csv

WHY THIS STEP MATTERS

For real time-series data, prefer a chronological split to avoid leakage.

CODE / TERMINAL COMMANDS

```
train, test = train_test_split(
df, test_size=0.25, random_state=42
)
train.to_csv("data/processed/train.csv", index=False)
test.to_csv("data/processed/test.csv", index=False)
```

NEXT → Train the Random Forest.

5. Train the model

Scikit-learn learns from two inputs

train.py loads train.csv and selects co2_ppm and pir_motion. It trains a Random Forest regressor and saves it so we can reuse it without retraining.

FILES / INPUTS / OUTPUTS

Python: src/train.py | Output: models/model.joblib

WHY THIS STEP MATTERS

The model file is the artifact later loaded by the API.

CODE / TERMINAL COMMANDS

```
X = df[["co2_ppm", "pir_motion"]]
y = df["people_count"]
model = RandomForestRegressor(**params)
model.fit(X, y)
joblib.dump(model, "models/model.joblib")
```

NEXT → Evaluate it on unseen data.

6. Set model parameters

Keep training settings outside the code

params.yaml stores the number of trees, tree depth and random seed. Changing these settings lets us compare experiments without rewriting the training script.

FILES / INPUTS / OUTPUTS

File: params.yaml

WHY THIS STEP MATTERS

DVC can detect parameter changes and rerun affected stages.

CODE / TERMINAL COMMANDS

```
random_forest:
n_estimators: 30
max_depth: 5
random_state: 42
```

NEXT → Measure model accuracy.

7. Evaluate predictions

MAE and RMSE describe model error

evaluate.py loads test.csv and model.joblib, predicts the people count, calculates errors and saves the metrics. Smaller errors generally indicate better predictions.

FILES / INPUTS / OUTPUTS

Python: src/evaluate.py | Output: metrics/metrics.json

WHY THIS STEP MATTERS

These metrics are also logged to MLflow for comparison.

CODE / TERMINAL COMMANDS

```
prediction = model.predict(df[FEATURES])
mae = mean_absolute_error(y, prediction)
rmse = mean_squared_error(y, prediction) ** 0.5
```

NEXT → Try one prediction directly.

8. Make a local prediction

Load the saved model

predict.py loads the trained model, builds a one-row DataFrame, predicts a number and rounds it to a nonnegative integer for the simple API.

FILES / INPUTS / OUTPUTS

Python: src/predict.py | Input: models/model.joblib

WHY THIS STEP MATTERS

We now have a useful artifact, but need to version the project.

CODE / TERMINAL COMMANDS

```
python -m src.predict
# Inside predict.py
model = joblib.load("models/model.joblib")
result = model.predict(row)[0]
```

NEXT → Introduce Git and GitHub.

9. Git: code versioning

Record what changed and when

Git tracks changes to scripts, configuration and small metadata files. You can return to an earlier commit if a change breaks the pipeline.

FILES / INPUTS / OUTPUTS

Files tracked: src/*.py, params.yaml, dvc.yaml

WHY THIS STEP MATTERS

Do not commit secrets, virtual environments or DVC-managed data blobs.

CODE / TERMINAL COMMANDS

```
git init
git add .
git commit -m "Initial people counting project"
git status
```

NEXT → Publish the code to GitHub.

MLOPS / PEOPLE COUNTING

10. GitHub: remote code repository

Keep code online and collaborate

GitHub hosts your Git repository. A push sends committed changes to GitHub, where teammates and GitHub Actions can access them.

FILES / INPUTS / OUTPUTS

Remote repository: people-counting-mlops

WHY THIS STEP MATTERS

GitHub stores code; it is not the same as a DVC data remote.

CODE / TERMINAL COMMANDS

```
git branch -M main
git remote add origin YOUR_GITHUB_URL
git push -u origin main
```

NEXT → Version the CSV separately with DVC.

MLOPS / PEOPLE COUNTING

11. DVC: data versioning

Track exactly which dataset was used

DVC records a version identifier for data/raw/people_count.csv. The small .dvc metadata file goes into Git; the dataset content can be sent to remote storage.

FILES / INPUTS / OUTPUTS

Input: data/raw/people_count.csv | Metadata: data/raw/people_count.csv.dvc

WHY THIS STEP MATTERS

Initialize Git before DVC.

CODE / TERMINAL COMMANDS

```
dvc init
dvc add data/raw/people_count.csv
git add .
git commit -m "Track dataset with DVC"
```

NEXT → Store DVC data remotely.

12. DagsHub: remote data storage

Share DVC-managed datasets and outputs

DagsHub can host the remote storage used by DVC. After configuring the remote and credentials, dvc push uploads data objects; dvc pull retrieves them.

FILES / INPUTS / OUTPUTS

Config: .dvc/config (shared remote URL)

WHY THIS STEP MATTERS

Store credentials locally or in secrets, never in Git.

CODE / TERMINAL COMMANDS

```
dvc remote add -d storage YOUR_DAGSHUB_DVC_URL
dvc push
# On another machine
dvc pull
```

NEXT → Connect preprocessing, training and evaluation.

MLOPS / PEOPLE COUNTING

13. DVC pipeline: dvc.yaml

Make the steps reproducible

dvc.yaml declares the command, dependencies and outputs for each stage. DVC uses this graph to decide which stages must rerun when data, code or parameters change.

FILES / INPUTS / OUTPUTS

File: dvc.yaml | Generated: dvc.lock

WHY THIS STEP MATTERS

The full starter file also declares deps, params, outs and metrics.

CODE / TERMINAL COMMANDS

```
stages:
prepare:
cmd: python -m src.prepare
train:
cmd: python -m src.train
evaluate:
```

NEXT → Run the entire pipeline with one command.

MLOPS / PEOPLE COUNTING

14. Execute the DVC pipeline

One command, three connected scripts

dvc repro checks the stage dependencies and executes the necessary steps. It generates processed CSVs, model.joblib and metrics.json.

FILES / INPUTS / OUTPUTS

Inputs: dvc.yaml, params.yaml | Outputs: processed CSV, model, metrics

WHY THIS STEP MATTERS

DVC may skip unchanged stages, which saves time.

CODE / TERMINAL COMMANDS

```
dvc repro
prepare -> train -> evaluate
# Show evaluation metrics
dvc metrics show
```

NEXT → Track the resulting experiment.

MLOPS / PEOPLE COUNTING

15. MLflow: experiment tracking

Compare model configurations

The evaluation script logs Random Forest parameters, MAE, RMSE, metrics.json and the trained model to MLflow. Each run becomes a record you can compare.

FILES / INPUTS / OUTPUTS

Python: src/evaluate.py | Local output: mlruns/

WHY THIS STEP MATTERS

MLflow tracks experiments; DVC tracks pipeline and data versions.

CODE / TERMINAL COMMANDS

```
mlflow.set_experiment("simple-people-counting")
with mlflow.start_run():
mlflow.log_params(params)
mlflow.log_metrics(metrics)
```

NEXT → Open the experiment interface.

MLOPS / PEOPLE COUNTING

16. View MLflow locally

See runs and compare errors

Start the MLflow UI and open it in your browser. Change params.yaml, rerun dvc repro, then compare the new metrics with the previous run.

FILES / INPUTS / OUTPUTS

Files: params.yaml, mlruns/

WHY THIS STEP MATTERS

Run the UI in one terminal and the pipeline in another.

CODE / TERMINAL COMMANDS

```
mlflow ui --port 5001
# Open in browser
http://127.0.0.1:5001
```

NEXT → Move experiment tracking to DagsHub.

MLOPS / PEOPLE COUNTING

17. DagsHub + MLflow

Track experiments remotely

DagsHub can provide a remote MLflow tracking endpoint. Configure its URL and credentials in the terminal so the same evaluate.py sends runs to that server.

FILES / INPUTS / OUTPUTS

Configuration: environment variables | Script unchanged: src/evaluate.py

WHY THIS STEP MATTERS

These are Windows Command Prompt commands; do not commit tokens.

CODE / TERMINAL COMMANDS

```
set MLFLOW_TRACKING_URI=YOUR_DAGSHUB_MLFLOW_URL
set MLFLOW_TRACKING_USERNAME=YOUR_USER
set MLFLOW_TRACKING_PASSWORD=YOUR_TOKEN
dvc repro -f evaluate
```

NEXT → Automate quality checks with tests.

MLOPS / PEOPLE COUNTING

18. Pytest: test your code

Catch problems before deployment

The starter test checks basic data-generation behavior. Tests help detect failures quickly when code is changed.

FILES / INPUTS / OUTPUTS

Python: tests/test_data.py

WHY THIS STEP MATTERS

A passing test does not guarantee that the ML model is scientifically accurate.

CODE / TERMINAL COMMANDS

```
python -m pytest -q
# Typical test structure
def test_example():
assert 2 + 2 == 4
```

NEXT → Run tests automatically on GitHub.

19. GitHub Actions: CI

Test automatically after a push

ci.yml runs on push and pull request. It checks out code, installs Python and dependencies, runs pytest, regenerates synthetic data and runs dvc repro.

FILES / INPUTS / OUTPUTS

File: .github/workflows/ci.yml

WHY THIS STEP MATTERS

CI is event-driven automation, unlike Airflow scheduling.

CODE / TERMINAL COMMANDS

```
git add .
git commit -m "Improve training"
git push
# GitHub -> Actions -> Test ML project
```

NEXT → Schedule regular pipeline runs.

20. Airflow: workflow orchestration

Schedule work and monitor task status

Airflow manages when a DAG runs, task order and failures. In this starter project, a single Airflow BashOperator generates data and calls the DVC pipeline.

FILES / INPUTS / OUTPUTS

Python: dags/people_counting_dag.py

WHY THIS STEP MATTERS

The Airflow container must have the project mounted and dependencies installed.

CODE / TERMINAL COMMANDS

```
schedule="0 2 * * *"
bash_command=(
"cd /usr/local/airflow/ml_project "
"&& python -m src.generate_data && dvc repro"
)
```

NEXT → Use Astro to run Airflow.

MLOPS / PEOPLE COUNTING

21. Astro: local Airflow environment

Start Airflow with Docker

Astro CLI creates and runs a local Airflow project. Copy the DAG into its dags/ folder and mount the ML project so the DAG can execute the scripts.

FILES / INPUTS / OUTPUTS

Files: astro/README.md | DAG: dags/people_counting_dag.py

WHY THIS STEP MATTERS

Astro runs Airflow; DVC still manages the ML stages.

CODE / TERMINAL COMMANDS

```
mkdir people-airflow
cd people-airflow
astro dev init
# Copy DAG and configure project mount
astro dev start
```

NEXT → Now make the model accessible through an API.

22. Flask: prediction API

Let another program request predictions

app.py exposes a health endpoint and a POST /predict endpoint. It reads two numbers, calls predict_people() and returns a JSON response.

FILES / INPUTS / OUTPUTS

Python: app.py, src/predict.py | Model: models/model.joblib

WHY THIS STEP MATTERS

The API needs the trained model file to exist first.

CODE / TERMINAL COMMANDS

```
python app.py
GET  /health
POST /predict
{ "co2_ppm": 760, "pir_motion": 7 }
```

NEXT → Package the API with Docker.

23. Dockerfile: package the API

Specify how to build a runnable image

Dockerfile chooses Python 3.11, installs API dependencies and copies app.py, src/ and model.joblib into the image. It starts the Flask server.

FILES / INPUTS / OUTPUTS

Files: Dockerfile, requirements-api.txt

WHY THIS STEP MATTERS

Docker image creation requires a trained model in the build context.

CODE / TERMINAL COMMANDS

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements-api.txt .
RUN pip install -r requirements-api.txt
COPY app.py .
COPY src/ ./src/
```

NEXT → Build and run the image.

24. Docker: build and run

Same packaged application across environments

docker build creates an image; docker run starts a container from it. Docker isolates the app dependencies from the host machine.

FILES / INPUTS / OUTPUTS

Files: Dockerfile, .dockerignore

WHY THIS STEP MATTERS

Stop any local Flask server using port 5000 first.

CODE / TERMINAL COMMANDS

```
docker build -t people-counting-api .
docker run --rm -p 5000:5000 people-counting-api
# Test in browser
http://127.0.0.1:5000/health
```

NEXT → Simplify startup with Docker Compose.

25. Docker Compose

Run containers using one configuration file

docker-compose.yml defines how to start the API. It is useful when a project grows to multiple services, but our starter runs just one.

FILES / INPUTS / OUTPUTS

File: docker-compose.yml

WHY THIS STEP MATTERS

Compose is a local orchestration convenience, not a cloud deployment.

CODE / TERMINAL COMMANDS

```
docker compose up --build
# Stop services
docker compose down
```

NEXT → Publish the image to a registry.

MLOPS / PEOPLE COUNTING

26. Docker Hub: image registry

Store and distribute Docker images

Docker Hub stores the built image so other machines can pull it. The CD workflow can publish the image after tests and training succeed.

FILES / INPUTS / OUTPUTS

Image: YOUR_USER/people-counting-api:latest

WHY THIS STEP MATTERS

Docker Hub stores images; it does not automatically run them.

CODE / TERMINAL COMMANDS

```
docker tag people-counting-api YOUR_USER/people-counting-api:latest
docker login
docker push YOUR_USER/people-counting-api:latest
```

NEXT → Automate image publication.

MLOPS / PEOPLE COUNTING

27. GitHub Actions: CD

Build and publish with a workflow

The included cd.yml is manually triggered. It runs tests, trains the model, logs into Docker Hub and builds/pushes the image using GitHub Actions.

FILES / INPUTS / OUTPUTS

File: .github/workflows/cd.yml | Secrets: DOCKERHUB_USERNAME, DOCKERHUB_TOKEN

WHY THIS STEP MATTERS

The included CD publishes an image; AWS deployment is a separate step.

CODE / TERMINAL COMMANDS

```
GitHub -> Settings -> Secrets and variables
GitHub -> Actions -> Publish Docker image
Click: Run workflow
```

NEXT → Run the published image on a server.

28. AWS EC2: cloud deployment

Host the prediction API online

An EC2 instance can pull the published Docker image and run the API. You must configure network access, security, updates and HTTPS appropriately.

FILES / INPUTS / OUTPUTS

Guide: docs/OPTIONAL_CLOUD.md

WHY THIS STEP MATTERS

Cloud services may incur costs. Restrict access during testing.

CODE / TERMINAL COMMANDS

```
docker pull YOUR_USER/people-counting-api:latest
docker run -d --restart unless-stopped \
-p 5000:5000 YOUR_USER/people-counting-api:latest
```

NEXT → Monitor the running service.

MLOPS / PEOPLE COUNTING

29. Monitoring: Grafana

See whether the service remains healthy

Grafana visualizes metrics collected by monitoring systems. Examples include request latency, error rate and changes in input CO2 or PIR values.

FILES / INPUTS / OUTPUTS

Potential files to add later: monitoring/

WHY THIS STEP MATTERS

The starter ZIP does not yet configure Grafana or a metrics collector.

CODE / TERMINAL COMMANDS

```
API request -> metrics collector -> Grafana
Track: latency, failures, input drift
With labels: MAE, RMSE
```

NEXT → Use monitoring evidence to decide on retraining.

30. Retraining and model updates

Close the MLOps feedback loop

When new labeled data arrives, update the CSV, version it with DVC, rerun the pipeline and compare the new model in MLflow. Only deploy if evaluation and tests pass.

FILES / INPUTS / OUTPUTS

Files: people_count.csv, dvc.yaml, params.yaml

WHY THIS STEP MATTERS

Do not deploy a worse model automatically without checks.

CODE / TERMINAL COMMANDS

```
python -m src.generate_data
dvc add data/raw/people_count.csv
dvc repro
dvc push
git add . && git commit -m "Update data"
```

NEXT → Review how the tools connect.

31. Which tool does what?

The five differences that matter most

Git tracks code; DVC tracks data and pipeline dependencies; MLflow tracks experiment results; Airflow schedules runs; GitHub Actions automates tests and image publication.

FILES / INPUTS / OUTPUTS

Files: .git/, .dvc/, dvc.yaml, mlruns/, dags/, .github/

WHY THIS STEP MATTERS

These tools complement rather than replace each other.

CODE / TERMINAL COMMANDS

```
Git:            git push
DVC:            dvc repro / dvc push
MLflow:         log_metric / log_model
Airflow:        DAG schedule
GitHub Actions: ci.yml / cd.yml
```

NEXT → Review the complete project map.

32. Full project file checklist

What to open when something goes wrong

Each stage has a main file. Reading these files in order is the easiest way to understand the starter project.

FILES / INPUTS / OUTPUTS

CSV -> Python -> configuration -> API -> CI/CD

WHY THIS STEP MATTERS

Some outputs are created after running dvc repro.

CODE / TERMINAL COMMANDS

```
DATA:       data/raw/people_count.csv
PREPARE:    src/prepare.py
TRAIN:      src/train.py + params.yaml
EVALUATE:   src/evaluate.py
PIPELINE:   dvc.yaml
AIRFLOW:    dags/people_counting_dag.py
```

NEXT → Finish with a short command checklist.

33. Command checklist

The practical path from ZIP to API

Run the model locally first. Then add GitHub, DagsHub, CI, Airflow, Docker and optional cloud deployment. This order makes troubleshooting much easier.

FILES / INPUTS / OUTPUTS

Start in the extracted project directory

WHY THIS STEP MATTERS

Each command is used at the appropriate stage, not all at once.

CODE / TERMINAL COMMANDS

```
conda activate people-mlops
python -m pytest -q
dvc repro
mlflow ui --port 5001
git push
dvc push
```

NEXT →

34. Summary: one connected story

From sensor measurements to a monitored API

The two-feature people-counting model stays simple. MLOps adds reproducibility, collaboration, automation and reliable operation around it. You can later replace synthetic data with real sensor measurements.

FILES / INPUTS / OUTPUTS

Core artifacts: CSV, .py, .yaml, model.joblib, Dockerfile

WHY THIS STEP MATTERS

Start with data and model correctness, then add infrastructure incrementally.

CODE / TERMINAL COMMANDS

```
DATA -> DVC -> PREPARE -> TRAIN -> EVALUATE
|
MLFLOW
AIRFLOW schedules; GITHUB ACTIONS tests
FLASK serves; DOCKER packages; AWS runs
```

NEXT → End of presentation.